Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: Riccardo Polo Discussed with: Giovanni Guidarini

Solution for Project 6

Due date: **Friday** 22 May 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Graph Partitioning with Julia: Load balancing for HPC [50 points]

1.1. Graph Partitioning Algorithms

1.1.1. Inertial Bisection algorithm

Listing 1 reports the implementation of the inertial bisection algorithm. This method uses the geometrical information associated with the graph vertices, namely their coordinates, to split the graph into two subdomains. The basic idea is to interpret the vertices as a set of points in the plane and to compute the direction along which the point cloud has the largest spatial spread. The graph is then partitioned by projecting the vertices onto this direction and separating them according to their position with respect to a cutting line.

Listing 1: Inertial Bisection algorithm

```
function inertial_part(A, coords)

    p = ones(Int, size(coords, 1))

    xbar = mean(coords[:, 1])
    ybar = mean(coords[:, 2])

    Sxx = sum((coords[:, 1] .- xbar) .^ 2)
    Syy = sum((coords[:, 2] .- ybar) .^ 2)
    Sxy = sum((coords[:, 1] .- xbar) .* (coords[:, 2] .- ybar))

    M = Symmetric([Syy -Sxy;
                   -Sxy Sxx])

    eigvals, eigvecs = eigs(M; nev=1, which=:SM)
    eigv = real.(eigvecs[:, 1])

    p1, p2 = partition(coords, eigv)

    p[p1] .= 1
    p[p2] .= 2

    return p
end
```

The function first initializes the partition vector `p`, assigning all vertices to the same subdomain. The number of entries of `p` is equal to the number of rows of `coords`, hence to the number of vertices of the graph.
The quantities `xbar` and `ybar` represent the coordinates of the center of mass of the vertices. The coordinates are then shifted with respect to this center of mass in order to compute the second-order moments `Sxx`, `Syy`, and `Sxy`. These quantities measure the spatial distribution of the vertices around the center of mass: `Sxx` and `Syy` quantify the spread along the coordinate directions, while `Sxy` measures the correlation between the two coordinates.

Using these moments, the matrix

$$M = \begin{pmatrix} S_{yy} & -S_{xy} \\ -S_{xy} & S_{xx} \end{pmatrix}$$

is assembled as a symmetric matrix. The eigenvector associated with the smallest eigenvalue of `M` defines the principal inertial direction of the point set. In the implementation, this eigenvector is computed by calling `eigs` with `nev=1` and `which=:SM`, which selects the eigenpair of smallest magnitude. Since the matrix is real and symmetric, the resulting eigenvector is converted to its real part and stored in `eigv`.
The actual split is then performed by the auxiliary routine `partition(coords, eigv)`. This routine projects the vertex coordinates onto the inertial direction and separates the vertices into two sets, denoted by `p1` and `p2`. These two sets correspond to the two sides of the bisecting line. Finally, the partition vector is updated by assigning label `1` to all vertices in `p1` and label `2` to all vertices in `p2`. The returned vector `p` therefore contains the bisection of the graph induced by the inertial direction.

1.1.2. Spectral Bisection Algorithm

In Listing 2, we report the implementation of the spectral bisection algorithm. Unlike inertial bisection, this method does not use the geometrical coordinates of the vertices. Instead, it exploits the connectivity structure of the graph through the graph Laplacian matrix. The main idea is to compute an eigenvector of the Laplacian, known as the Fiedler vector, and use its components to divide the vertices into two groups.

Listing 2: Spectral Bisection algorithm

```
function spectral_part(A)

    n = size(A, 1)

    if n == 1
        return ones(Int, 1)
    end

    # 1. Construct the Laplacian matrix.
    D = Diagonal(vec(sum(A, dims=2)))
    L = Symmetric(Matrix(D - A))

    # 2. Compute eigendecomposition.
    eig = eigen(L)

    # 3. Take eigenvector associated with second smallest eigenvalue.
    perm = sortperm(eig.values)
    fiedler_vec = eig.vectors[:, perm[2]]

    # 4. Balanced partition by sorting the Fiedler vector.
    order = sortperm(fiedler_vec)

    p = ones(Int, n)

    n1 = div(n, 2)
    p[order[1:n1]] .= 1
    p[order[n1+1:end]] .= 2

    return p

end
```

The function takes as input the adjacency matrix `A` of the graph and first extracts the number of vertices `n`. A special case is included for graphs with a single vertex: in this situation no meaningful bisection can be performed, and the function simply returns a partition vector containing one label.

The first step of the algorithm consists in constructing the graph Laplacian. The diagonal matrix `D` contains the degrees of the vertices, computed by summing the rows of the adjacency matrix. The Laplacian is then assembled as

$$L = D - A.$$

In the implementation, the matrix is explicitly converted to a symmetric matrix using `Symmetric(Matrix(D - A))`. This is consistent with the fact that, for an undirected graph, the Laplian is real and symmetric.
The spectral information of the graph is then obtained by computing the eigendecomposition of `L`. The eigenvalues are sorted in increasing order, and the eigenvector associated with the second smallest eigenvalue is selected. This vector is the Fiedler vector. The smallest eigenvalue of the Laplacian is associated with the constant eigenvector and does not provide a useful separation of the graph. The second smallest eigenvector, instead, contains information about the weakest connected direction of the graph and is therefore used to define the bisection.

In the theoretical version of spectral bisection, the split can be obtained by comparing the components of the Fiedler vector with a threshold, often chosen as zero or as the median value. In this implementation, a balanced partition is enforced explicitly. The entries of the Fiedler vector are sorted, and the first

`div(n, 2)` vertices in this ordering are assigned to the first subdomain, while the remaining vertices are assigned to the second subdomain. This guarantees that the two parts have approximately the same number of vertices, which is desirable in recursive graph partitioning because it avoids strongly unbalanced subdomains.

Finally, the partition vector `p` is returned. Its entries contain the labels of the two subdomains: vertices assigned label `1` belong to the first part, while vertices assigned label `2` belong to the second part.

1.1.3. Results

The script `main_bench.jl` was used to run the four bisection strategies on the toy meshes loaded by `bench_bisection.jl`. For each mesh and each partitioning strategy, we measured the bisection edge-cut, i.e. the number of graph edges whose endpoints belong to different parts. Since the partitions are constrained to be balanced, the edge-cut provides a simple measure of the quality of the interface between the two subdomains: lower values correspond to fewer inter-partition connections and therefore to less communication in a parallel graph-distributed computation.

Mesh	Coordinate	Inertial	Spectral	METIS
`3elt`	–	–	–	–
`airfoil1`	–	–	–	–
`barth4`	–	–	–	–
`crack`	–	–	–	–
`mesh1e1`	–	–	–	–
`mesh2e1`	–	–	–	–
`mesh3e1`	–	–	–	–
`netz4504_dual`	–	–	–	–
`stufe`	–	–	–	–
`ukerbe1`	–	–	–	–

Table 1: Bisection edge-cut obtained on the toy meshes loaded by `bench_bisection.jl`.

The corresponding partitions are shown in Figures 1–4 for the first four meshes, in Figures 5–9 for the additional structured meshes, and in Figure 10 for the `ukerbe1` mesh. The coordinate and inertial methods are purely geometrical: they only use the position of the vertices and therefore tend to produce almost straight cuts. This behaviour is clearly visible in the `3elt`, `barth4`, and `crack` examples. Such partitions are simple and often reasonable when the mesh is approximately uniform and the graph connectivity follows the geometry closely. However, they can become sub-optimal when the mesh has non-uniform density or when the geometry contains local features that should not be cut through.

The spectral and METIS partitions are instead based on the graph connectivity. As a consequence, they are generally more flexible: the interface is not restricted to be a straight geometrical line, but can follow weaker connections in the graph. This is particularly important for meshes such as `airfoil1`, where the local refinement around the airfoil and its wake makes a purely geometrical split less adapted to the graph structure. In this case, the coordinate and inertial cuts separate the mesh in a rather direct left-right fashion, while spectral bisection and METIS produce more irregular interfaces that better reflect the connectivity of the mesh.

Some surprising or clearly sub-optimal partitions can also be observed. In particular, for `barth4` the METIS and spectral results appear very unbalanced visually, with one part concentrated around only a small region near the centre of the mesh. A similar behaviour appears for the `3elt` mesh in the METIS and spectral plots, where one color occupies almost the entire mesh and the other is concentrated in a very small central region. This suggests that either the partition labels are not being displayed as expected, or that the returned partition is not a proper balanced bisection for these cases. Since recursive bisection relies on each split producing two meaningful subdomains, such cases should be checked carefully by verifying the number of vertices assigned to each part in addition to the edge-cut value.

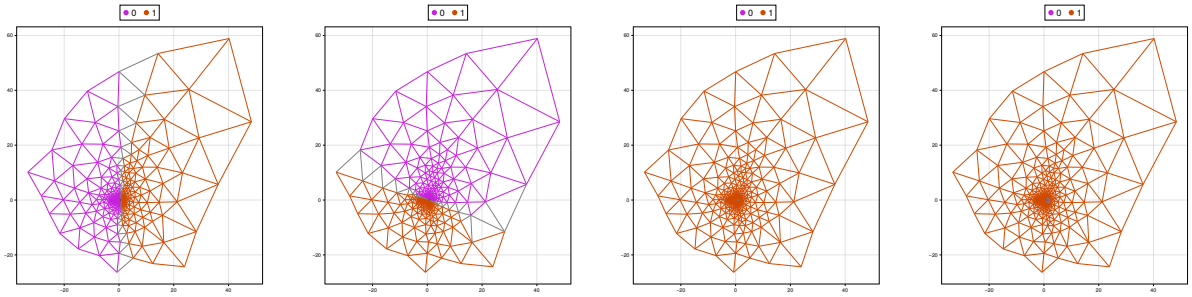
The additional structured meshes further illustrate the difference between geometrical and connectivity-based partitioning. For the regular meshes `mesh1e1`, `mesh2e1`, and `mesh3e1`, all methods produce relatively clean partitions, but the shape of the interface depends on the criterion used by the algorithm. Coordinate bisection typically produces a vertical or horizontal cut aligned with the coordinate axes, while inertial bisection may rotate the cut according to the dominant spatial direction of the mesh. Spectral bisection and

METIS may instead introduce slightly irregular interfaces when this reduces the number of cut edges.

The `netz4504_dual` and `stufe` meshes are more interesting because their geometry is less uniform. In these cases, a purely geometrical cut does not necessarily correspond to the smallest communication interface. The graph-based methods are able to adapt the separator to the local connectivity of the mesh, especially around refined or stepped regions. This is the expected behaviour: the objective of graph partitioning is not to produce a visually straight cut, but to minimize the number of edges crossing between different parts while keeping the load balanced.

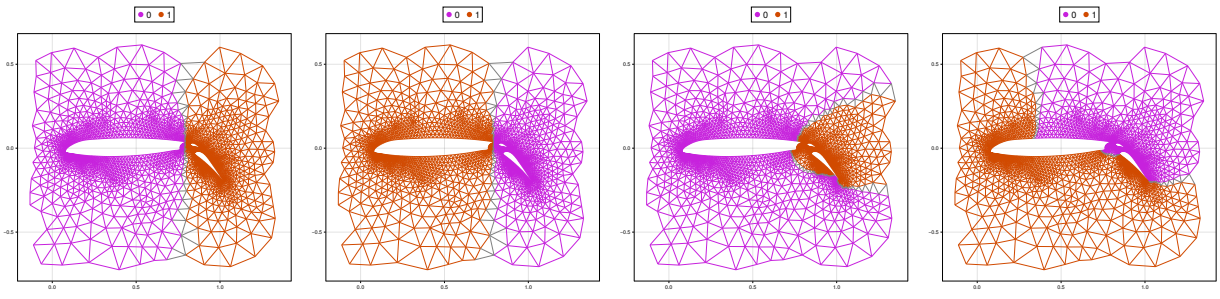
Finally, the `ukerbe1` mesh shows a geometry with a pronounced notch-like feature and several internal circular holes. In this case, the coordinate and inertial methods still produce mostly straight cuts, separating the mesh approximately into upper and lower parts. This is reasonable from the point of view of the global geometry, but it does not explicitly account for the connectivity around the notch and the locally refined region. The spectral and METIS partitions instead place the separator very close to this narrow central region. This behaviour is consistent with the graph-based objective, since cutting through a geometrical bottleneck can reduce the number of edges crossing between the two parts. However, the METIS partition appears visually suspicious, since one part is concentrated in a very small region close to the notch while the rest of the mesh belongs to the other part. As for the previous anomalous cases, this should be checked against the actual part sizes printed by the script, because a low edge-cut is meaningful only if the balance constraint is also satisfied.

Overall, the results confirm the expected trade-off between geometrical and graph-based partitioning. Coordinate and inertial bisection are cheap and produce simple cuts, but they may ignore important connectivity features. Spectral bisection and METIS can produce lower edge-cuts because they exploit the graph structure directly, but the resulting partitions may be less regular geometrically and, in some cases, should be inspected to ensure that the balance constraint has been respected.



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 1: Bisection partitions obtained for the `3elt` mesh.



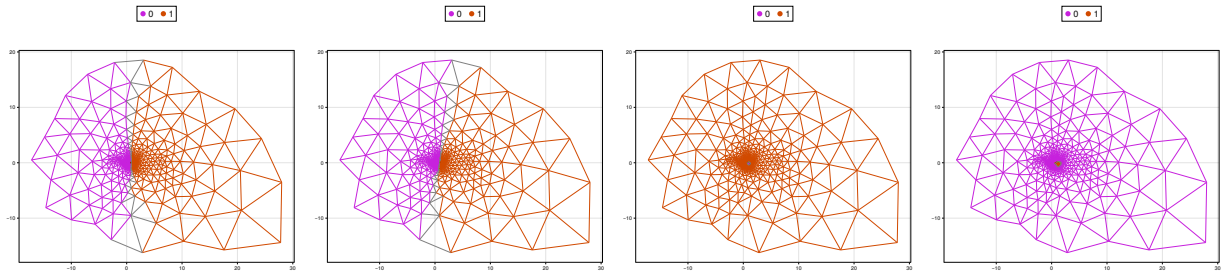
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 2: Bisection partitions obtained for the `airfoil1` mesh.

1.2. Recursive Bisection Algorithm

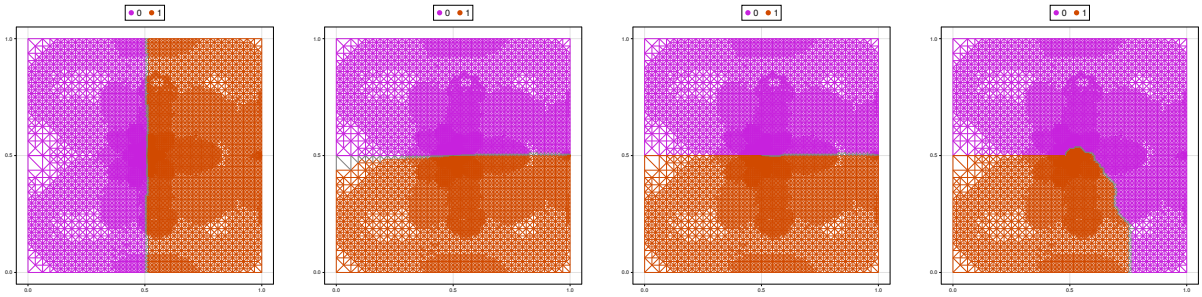
1.2.1. Implementation

In lis



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 3: Bisection partitions obtained for the `barth4` mesh.



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 4: Bisection partitions obtained for the `crack` mesh.

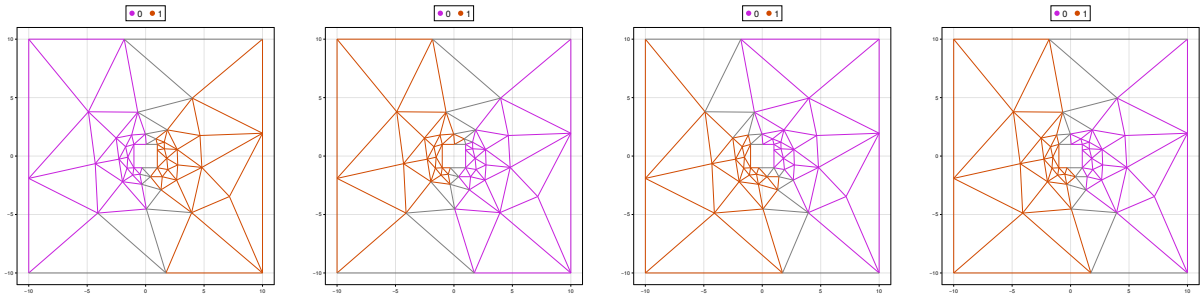
2. HPC software frameworks [50 points]

2.1. Mathematical Formulation

2.2. PETS Implementation

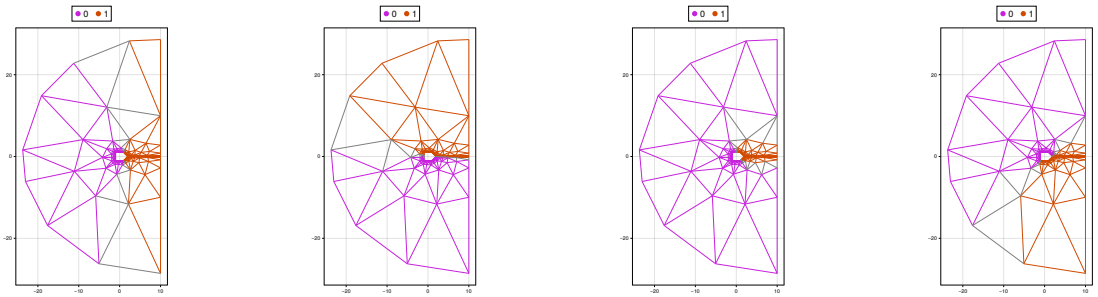
2.3. Visualization

2.4. Strong Scaling Analysis



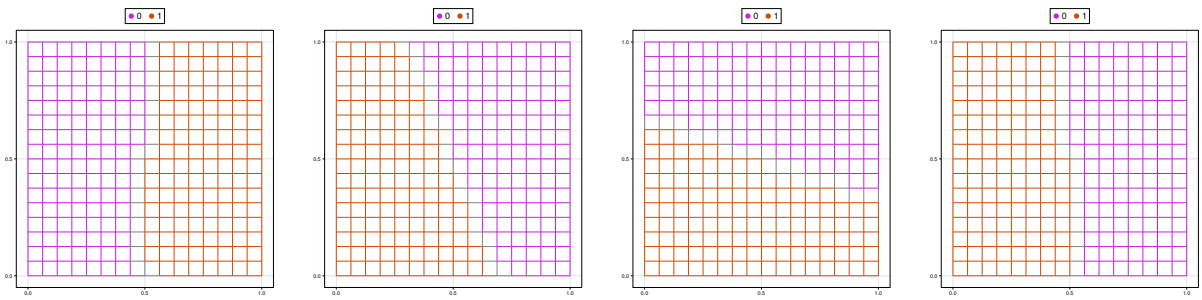
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 5: Bisection partitions obtained for the `mesh1e1` mesh.



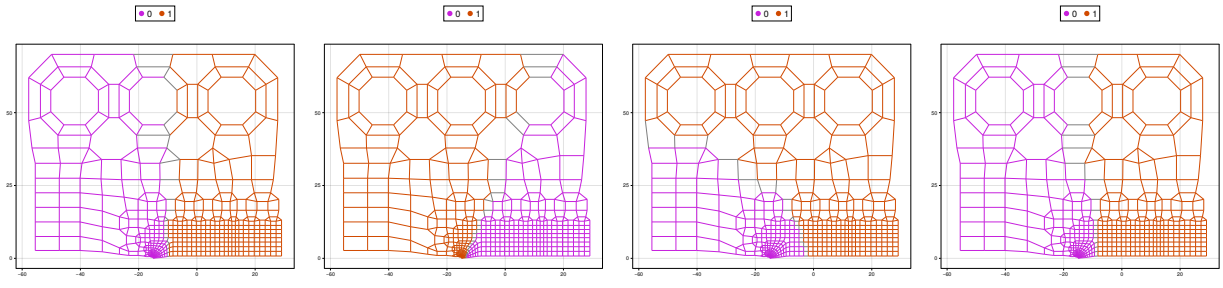
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 6: Bisection partitions obtained for the `mesh2e1` mesh.



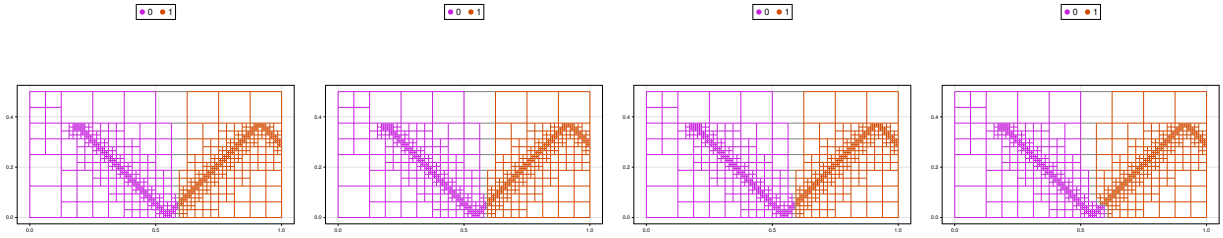
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 7: Bisection partitions obtained for the `mesh3e1` mesh.



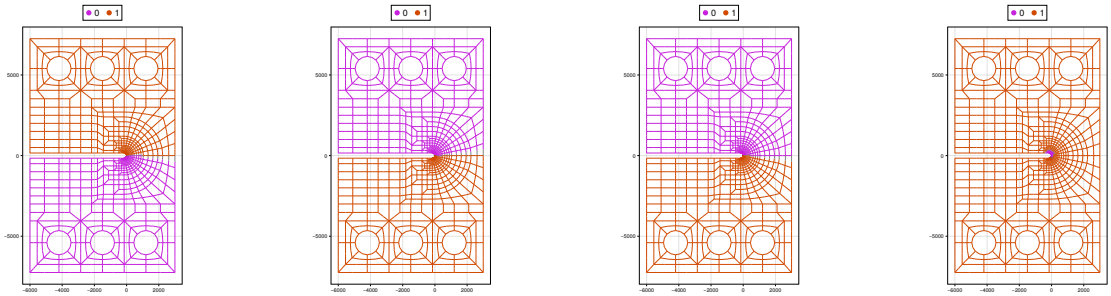
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 8: Bisection partitions obtained for the `netz4504_dual` mesh.



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 9: Bisection partitions obtained for the `stufe` mesh.



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 10: Bisection partitions obtained for the `ukerbe1` mesh.